



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

IEE2714-1

Tarea 2

Fecha de entrega:
9 de Octubre De 2026
Estudiante:
Renato Salinas

Link github repository:

<https://github.com/The2troller/imagenes/tree/main/tarea%202>

Explicaciones:

Pregunta 1:

Según la distribución Poisson solicitada, el factor de intensidad multiplicado por 40 es la variable de ingreso de la función `np.random.poisson lamb`, lo que significa un valor de intensidad mayor proyectara una mayor variación final.

Como se puede observar en la fig 2 la variación en intensidades iniciales de las distintas máscaras asociadas a el fondo (0.15), el cuadrado (0.45) y el círculo (0.8) tomando un valor de filtro nulo, se aprecia que, siguiendo el mismo orden, obtenemos valores iniciales de RMSE de aproximadamente 0.06, 0.11 y 0.14, lo que nos permite confirmarlo. Esto además nos provoca que el mismo σ no minimice el error en las 3 regiones, puesto que al tener magnitudes de ruido tan distintas, cada región requiere un grado de suavizado espacial diferente para alcanzar su punto óptimo.

Se incluyen en las máscaras los píxeles próximos a los bordes para exponer el error por desenfoque y no únicamente el error por ruido. Si con el RMSE se midiera sólo en una pequeña zona plana muy alejada de cualquier frontera el σ óptimo tendería a infinito debido a que, cada vez que se difumina más bajaría el RMSE, y al no tener bordes cercanos, no existiría error por difuminación, entonces no tendría ninguna desventaja el aumentar el σ .

El tamaño del kernel se determina a partir de la siguiente fórmula:

$size = \text{int}(np.ceil(sigma * 2)) * 2 + 1$ la cual es la cantidad de size mínima, siendo esta de 2σ , y al añadir un 1 conseguimos que el size sea necesariamente impar, lo que es un requisito para tener un pixel central, esto le permite al kernel usar una gran parte de la campana de gauss. A pesar de abarcar una gran cantidad, se termina perdiendo otra parte puesto que es imposible abarcarla completamente debido a su naturaleza infinita, lo que provocará que la imagen filtrada pierda intensidad, puesto que la suma de los coeficientes dentro de la matriz será estrictamente menor a 1.0 por la pérdida mencionada. Por lo que normalizar el kernel según los valores que estamos abarcando para que estos sumen 1.0 nos permite lograr que la imagen no pierda intensidad.

Se calcula el $\mu(x,y)$ a partir de la imagen original pasada por un filtro gaussiano con un sigma auxiliar el cual se explicita al llamar al objeto, en este caso establecido en 1.6 al ser el valor con menor RMSE, partir de este mapa suavizado, definimos una regla de asignación por tramos para obtener $\sigma(x,y)$, la cual es separar en máscaras según rangos de intensidad, considerado en este caso el hardcoded de los rangos dentro de los cuales deberían estar las secciones. Por otra parte, la imagen original se le aplican otras máscaras gaussianas usando como sigma el valor del mejor posible según RMSE para aquella máscara, para obtener 3 imágenes, las cuales finalmente uniremos en una según las máscaras calculadas previamente. Este método en fronteras aparecen halos debido a la difuminación de la imagen debido a que el trazo de las máscaras es posterior a la filtración base de la imagen, y transiciones abruptas en $\sigma(x,y)$ debido a la regla de asignación aplicada, la cual trabaja con escalones entre las regiones. Esto podría ser solucionado aplicando una interpolación suave, para que $\sigma(x,y)$ no se comporte como una función por escalones.

Finalmente se procedió a obtener los valores de $\mu(x,y)$, $\sigma(x,y)$ y kernel aplicado en 3 puntos, uno en cada uno de las máscaras. Del pixel de fondo (30, 30) se obtuvo $\mu \approx 0.1521$, $\sigma = 1.8$ y kernel gaussiano isotrópico de tamaño 9 por su lado, dentro del cuadrado (80, 80) se obtuvo $\mu \approx 0.4408$, $\sigma = 1.4$ y kernel gaussiano isotrópico de tamaño 7, finalmente dentro del círculo (128, 128) se obtuvo $\mu \approx 0.7991$, $\sigma = 1.4$ y kernel gaussiano isotrópico de tamaño 7.

Puesto que para que sea realmente adaptativo tiene que tener la posibilidad de alternar el kernel según el píxel dentro del cual se localiza, pero en el caso de una única convolución con kernel fijo, el kernel obviamente no cambia. Se puede aproximar a uno adaptativo de otra manera, la cual es mediante la que adaptamos dentro del código.

Al comparar el mejor gaussiano global ($\sigma = 1.6$) contra su versión adaptativa, el primero obtuvo un RMSE de 0.0756 y el segundo un RMSE de 0.0751, esto es debido a que el adaptativo tiene la capacidad de optimizar el σ por zona, obteniendo una imagen final lo mayor optimizada posible, igualmente, el adaptativo supera al global en el mantenimiento de bordes debido a su incorporación de imagen final a partir de máscaras calculadas previamente con otro σ . Pero por otro lado, esta mejora en bordes puede terminar perjudicando al adaptativo, ya que puede recurrir a pensar que existe un borde en algún lugar en el que realmente no hay, lo que provocará irregularidades, cosa que para el global no ocurre.

Pregunta 2:

En cada iteración las diferencias direccionales se calculan al inicio del ciclo utilizando la versión actual modificada de la imagen, se usa `np.roll` para calcular las gradientes eficientemente, y el

c y el laplaciano se calculan según el modo a utilizar, dentro del if para iniciar el modo, para el modo VT no se calcula literalmente el laplaciano, pero se siguen calculando las gradientes asociadas.

Específicamente en VT el papel de epsilon es de generar un umbral de tolerancia que separa el ruido local de los bordes estructurales, cuando el gradiente tiende a cero, pasa a tener el mayor peso epsilon dentro de la ecuación, cuando es muy chico, cada pequeñísimo cambio (gradiente) se interpreta como un borde, por lo que tiende a disminuir la difusión, al contrario cuando es muy grande tiende a no ver claramente los bordes, lo que provoca que se suavice la imagen como si fuese una, esto se puede apreciar más claramente en las en la figura 6 de mapas de c . Se experimentó con varios valores de epsilon, y se confirmó nuevamente que el epsilon mayor subestima el ruido y provocaba que el ruido no se lograra limpiar correctamente, el epsilon muy alto provocó una sobre suavización la que provocó que la imagen quedase con bordes difusos. El valor máximo de difusión no fue estimado, ya que como se normalizo la imagen este se vuelve 1.0 obligatoriamente, esto provoca que el criterio de estabilidad utilizado sea la condición de Courant-Friedrichs-Lewy, limitando λ a un valor máximo de 0.25, debido a que trabajamos con valores normalizados, por lo que al interactuar con 4 vecinos el máximo debiese de ser 1 y por lo tanto cada uno con un máximo de 0.25, como podemos observar en las figuras 12, 13 y 14, ocurren errores, el más claro es en G + LP, en donde se altera completamente la imagen quedando irreconocible.

Para las funciones $h(E)$, comenzando por el LP, cuando E tiende a 0, osea en zonas de intensidad constante, c tiende a 1.0, lo que significa una difusión máxima, en cambio cuando E aumenta el exponente crece en magnitud hacia el lado negativo, provocando que c tienda a 0, deteniendo la difusión, el parámetro que controla el umbral de estas divisiones, como se mencionó anteriormente es el epsilon. Continuando con el G + LP, las consecuencias de E tendiendo a 0 y aumentando son exactamente las mismas, y se seleccionan las divisiones siguiendo el mismo umbral epsilon.

Dentro de LP, la segunda derivada captura la curvatura, frente al ruido esto provoca que sea hipersensible y aumente el ruido en gran medida, en este modo no se implementaron medidas que contrarresten a este problema, las cuales fueron adoptadas en G + LP mediante una actuación más secundaria por parte del laplaciano

Si se llegase a reemplazar el coeficiente de difusión por una constante espacial y temporal, el algoritmo perdería por completo su capacidad adaptativa. La difusión se aplicaría con la misma fuerza en todas las direcciones y en todos los píxeles, ignorando la presencia de bordes, texturas o ruido.

Cuando se prueba una cantidad de iteraciones iguales a 5, se aprecia una evolución demasiado lenta, lo que provoca una imagen final idéntica a la imagen borrosa (figura 9). En cambio con una cantidad de iteraciones iguales a 100 (figura 9), la imagen se vuelve sobre suavizada y se pierden los bordes.

Para los dos coeficientes LP y G + LP, sus variables son las mismas que para el VT y su variable de mayor relevancia es el epsilon, sin embargo, para el primero, como expresa una

mayor sensibilidad, tiende a tener bordes más bruscos y firmes, y casualmente dentro del ruido, sin considerar bordes también, esto último es mitigado por el mecanismo del segundo, pero a cambio perdiendo ligeramente la capacidad de tener bordes nítidos.

Al comparar TV con LP y G + LP en la figura 18 y 19, se pueden apreciar diferencias claras en espacios claves, en los cuales LP y G + LP marcan los bordes con mayor efectividad, pero en gran escala, en esta imagen se aprecia que tiene problemas en la cabeza y otros lugares, los cuales terminan por difuminar excesivamente y no permiten mantener los bordes con calidad. Teóricamente hablando el modo LP y G + LP debieran de tener una mejor calidad a la hora de marcar bordes y por el contrario el modo TV una mejor calidad a la hora de difuminar zonas con intensidad similar.

Explicitando las ventajas y desventajas de cada c aplicado, para el TV, se tiene una mejor difuminación en zonas de igual intensidad pero a costa de bordes menos precisos, el LP, tiene una gran sensibilidad lo que provoca que tenga bordes bien marcados pero le cuesta difuminar dentro de áreas de intensidad más similar, finalmente el G + LP sería el punto medio entre ambos, ya que tiene la utilización del laplaciano pero diluido para bajar la sensibilidad.

Finalmente se muestran los datos obtenidos de las diferencias direccionales utilizadas, el valor de c y la actualización resultante en ambos casos.

```
--- DATOS ITERACIÓN 1 (Modo 1) ---  
Píxel Homogéneo en (100, 160):  
  Dif. N: -0.07021 | S: 0.00748 | E: -0.05194 | O: -0.06974  
  Valor de c: 0.03336  
  Actualización: -0.00302  
  
Píxel Borde en (172, 189):  
  Dif. N: -0.00974 | S: 0.04960 | E: 0.07937 | O: 0.03608  
  Valor de c: 0.08961  
  Actualización: 0.00605
```

```
--- DATOS ITERACIÓN 1 (Modo 2) ---  
Píxel Homogéneo en (100, 160):  
  Dif. N: -0.07021 | S: 0.00748 | E: -0.05194 | O: -0.06974  
  Valor de c: 0.89450  
  Actualización: -0.01460  
  
Píxel Borde en (172, 189):  
  Dif. N: -0.00974 | S: 0.04960 | E: 0.07937 | O: 0.03608  
  Valor de c: 0.98971  
  Actualización: 0.01139
```

Experimentación y análisis:

Pregunta 1:

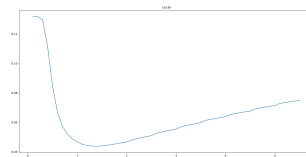


Fig 1. Gráfico de RMSE Imágen completa

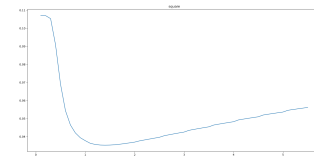
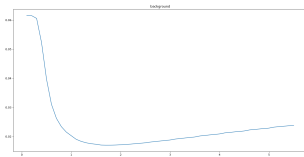
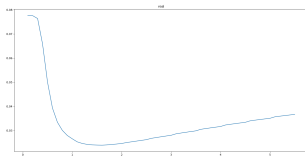


Fig 2. Gráfico de RMSE máscara aplicada fondo / cuadrado / círculo

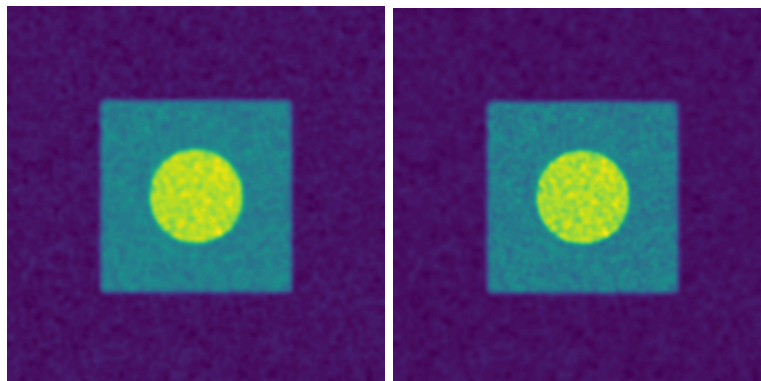


Fig 3. Imagen filtrada global vs adaptativamente



Fig 4. Mapa de σ aplicado

Pregunta 2:



Fig 5. Img original vs ruido aplicado

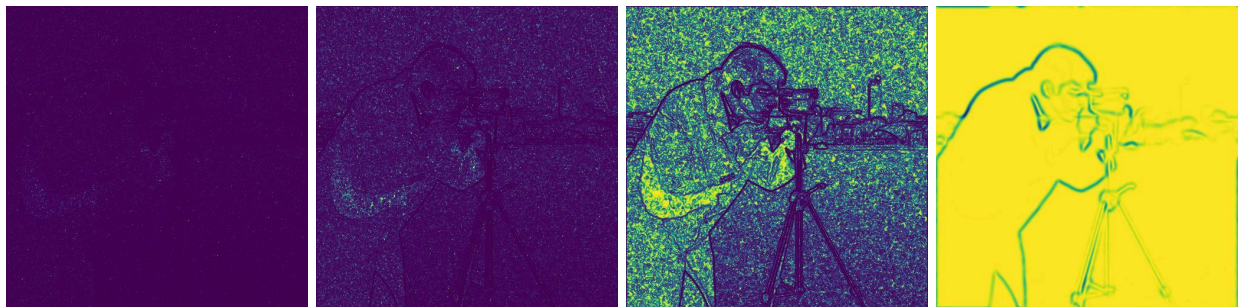


Fig 6. Mapa de coeficientes TV con epsilon 10^k con $k = -3.5 / -3.0 / -2.5 / -1$

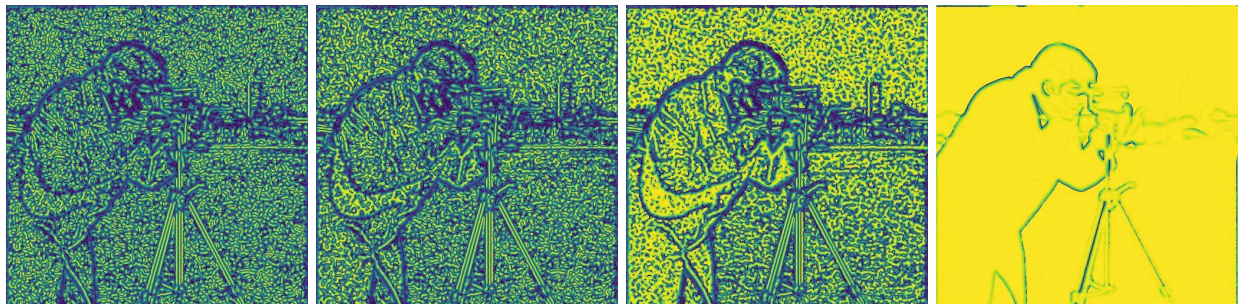


Fig 7. Mapa de coeficientes LP con epsilon 10^k con $k = -3.5 / -3.0 / -2.5 / -1$

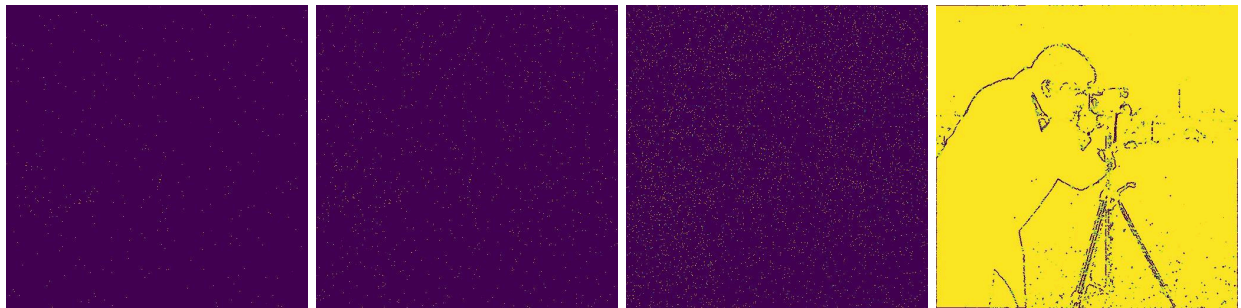


Fig 8. Mapa de coeficientes G + LP con epsilon 10^k con $k = -3.5 / -3.0 / -2.5 / -1$

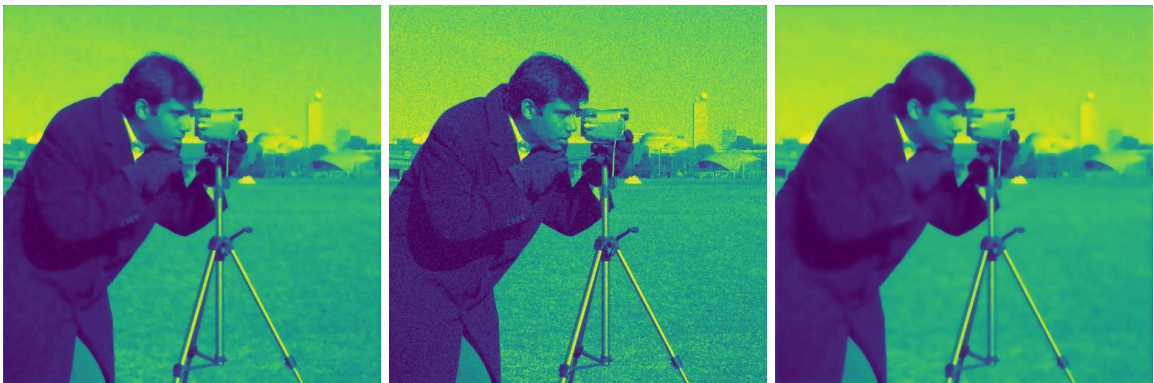


Fig 9. Imagen TV con núm iteraciones 50 / 5 / 100

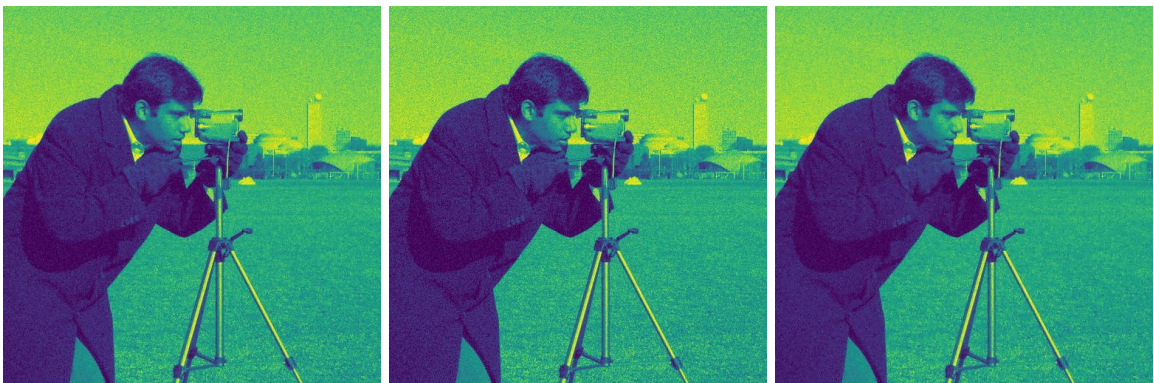


Fig 10. Imagen LP con núm iteraciones 50 / 5 / 100



Fig 11. Imagen G + LP con núm iteraciones 50 / 5 / 100



Fig 12. Imagen TV con lambda 0.1 / 0.01 / 0.25 / 0.35



Fig 13. Imagen LP con lambda 0.1 / 0.01 / 0.25 / 0.35



Fig 14. Imagen G + LP con lambda 0.1 / 0.01 / 0.25 / 0.35



Fig 15. Imagen TV con epsilon 10^k con $k = -3 / -2 / -3.5$

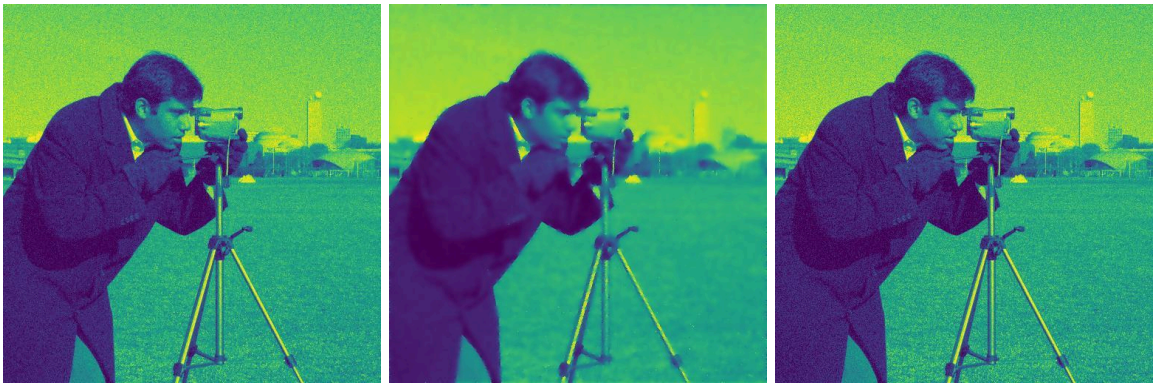


Fig 16. Imagen LP con epsilon 10^k con $k = -3 / -2 / -3.5$



Fig 17. Imagen G + LP con epsilon 10^k con $k = -3 / -2 / -3.5$



Fig 18. Comparación métodos TV vs LP vs G + LP



Fig 18. Comparación métodos TV vs LP vs G + LP (zoom)

Referencias: